

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3568 · Minimum Moves to Clean the Classroom

LeetCode · Weekly Contest 452

Medium

BFS

State Compression

1. What Type of Question Is This?

This is a **grid shortest-path problem with a resource constraint and multiple pickup targets**. It combines three classic ideas into one: (1) BFS on a grid, (2) tracking a consumable/rechargeable resource (energy) as part of the search state, and (3) tracking which of several items have been collected using a **bitmask**. Because the state is no longer just "which cell am I on", the search has to happen over an **augmented state space**: (row, column, remaining energy, collected-litter bitmask). Whenever you see "minimum steps to collect all of several items scattered on a grid", think **BFS + bitmask**.

2. What Is Given?

- A grid **classroom** of size $m \times n$ made of characters:

'S'	Student's starting cell (exactly one in the grid)
'L'	Litter to collect — becomes empty once visited (at most 10 in the grid)
'R'	Reset area — refills energy to full, reusable any number of times
'X'	Obstacle — cannot be entered
'.'	Empty walkable cell

- An integer **energy** — the student's maximum energy capacity. The student starts at 'S' with this much energy already full.
- Every move to an adjacent cell (up/down/left/right) costs **1 unit of energy**.
- If energy hits 0, the student is stuck **unless** they are standing on an 'R' cell, which instantly refills energy back to the maximum capacity.

3. What Do We Need to Find?

The **minimum number of moves** needed to collect every 'L' cell in the grid, or **-1** if it can never be done (e.g. an 'L' is unreachable, or energy always runs out before reaching the next 'L' or the next 'R').

Function Signature

■ C++

```
class Solution {
public:
    int minMoves(vector<string>& classroom, int energy) {

    }
};
```

Examples Walkthrough

Example 1

```
Input:  classroom = ["S.", "XL"], energy = 2
Output: 2
```

Grid: row 0 is **S** . and row 1 is **X L**. The student starts at (0,0) with 2 energy. Move 1: (0,0) → (0,1), 1 energy used, 1 remaining. Move 2: (0,1) → (1,1), collects the only litter. All litter collected in **2 moves** — energy never runs out, and no reset was even needed.

Example 2 — used as the running example for every code sample below

```
Input:  classroom = ["LS", "RL"], energy = 4
Output: 3
```

The student starts at (0,1) with 4 energy. Move 1: (0,1) → (0,0), collects the first 'L', using 1 energy (3 left). Move 2: (0,0) → (1,0), which is an 'R', restoring energy back to 4. Move 3: (1,0) → (1,1), collects the second 'L'. Total = **3 moves**. Every C++/Python/JavaScript snippet in this document runs this exact example through a small driver / **main()** function so you can copy-paste and see **3** printed out.

Example 3 (Impossible Case)

```
Input:  classroom = ["L.S", "RXL"], energy = 3
Output: -1
```

Grid: row 0 is **L . S**, row 1 is **R X L**. From S at (0,2), the student can reach the first 'L' at (0,0) in 2 moves. But the second 'L' at (1,2) is boxed in by the obstacle 'X' at (1,1) and there is no other route around it in this 2×3 grid — it is permanently unreachable, so the answer is **-1**. This shows why reachability (via BFS/flood fill) matters just as much as energy accounting.

Constraints

- $1 \leq m == \text{classroom.length} \leq 20$, $1 \leq n == \text{classroom}[i].\text{length} \leq 20$
- $\text{classroom}[i][j]$ is one of 'S', 'L', 'R', 'X', '.'
- $1 \leq \text{energy} \leq 50$
- There is exactly one 'S' in the grid
- There are at most 10 'L' cells in the grid

The "**at most 10 litter cells**" constraint is the biggest hint in the whole problem — it screams **bitmask** ($2^{10} = 1024$ possible collected-so-far states, small enough to brute-force over).

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Identify it's shortest-path.** All moves cost exactly 1, so "minimum moves" on a grid is a textbook **BFS** signal (BFS explores level by level, so the first time you reach a goal state, that's the shortest number of moves — no Dijkstra/priority-queue needed since edge weights are uniform).
- **Step 2 — Notice plain (row, col) BFS is not enough.** Two different paths can reach the exact same cell but with different amounts of energy left and different sets of litter already collected — and that difference changes what's possible from there on. So "has this cell been visited" is the wrong question; the real question is "has this exact *situation* been visited".
- **Step 3 — Expand the state.** A situation = (row, col, remaining energy, which litter is already collected). The last piece is naturally a **bitmask** since there are at most 10 litter cells.
- **Step 4 — Run BFS over that 4-part state** instead of over plain cells. Every state you dequeue that already has a full bitmask is your answer (first time it happens, by BFS's level-order guarantee, is the minimum).
- **Step 5 — Handle the energy rule as a transition rule**, not a separate search: you can only move if energy > 0 (after spending 1 for the move); landing on 'R' resets energy to the max regardless of what it was.
- **Step 6 — If BFS empties out** without ever reaching a full bitmask, return -1.

Which Algorithm Is Best?

BFS over the augmented state space (row, col, energy, bitmask) is the intended and asymptotically optimal approach — this is exactly what LeetCode's own editorial explanation uses. There isn't a fundamentally different, faster algorithm family for this problem (it isn't a shortest-path-with-weights problem, so Dijkstra/A* add nothing; it isn't a pure TSP, because energy can't be decoupled from the order of visits, since it only resets at specific cells). What *does* change between the three tiers in this document is how efficiently that same core idea is implemented: no state tracking at all (brute force) → correct state tracking with BFS (better) → the same BFS with a compact flattened encoding and an early-exit check (optimal).

Approach 1 — Brute Force (DFS, No State Tracking)

Idea: Try every possible sequence of moves from the start using plain recursion. Whenever all litter is collected, compare the move count against the best found so far. Never remember a visited state, so the same (cell, energy, mask) combination can be explored over and over from different paths.

Steps

- Parse the grid: locate 'S', and assign each 'L' cell a bit index (0, 1, 2, ...).
- Recursively explore all 4 directions from the current cell.
- Skip a move if the target cell is 'X', or if current energy is already 0.
- Decrease energy by 1 per move; if the target cell is 'R', reset energy to max instead.
- If the target cell is an uncollected 'L', flip its bit on in the mask.
- Whenever the mask becomes full, update the best (minimum) move count seen so far.
- Prune only when the current move count already exceeds the best answer found — this helps a little, but the search is still fundamentally exponential because paths can revisit the exact same situation many times.

Approach 1 — Code (all languages run Example 2)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int m, n, energyCap, full, best;
    vector<string> g;
    vector<vector<int>> bit;          // bit index for each 'L' cell, -1 otherwise
    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};

    void dfs(int r, int c, int e, int mask, int moves) {
        if (moves >= best) return;      // already worse than best answer found
        if (mask == full) { best = min(best, moves); return; }

        for (int d = 0; d < 4; d++) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
            if (g[nr][nc] == 'X') continue;
            if (e == 0) continue;      // stuck: no energy and not on 'R'

            int ne = e - 1;
            int nmask = mask;
            if (bit[nr][nc] != -1) nmask |= (1 << bit[nr][nc]);
            if (g[nr][nc] == 'R') ne = energyCap;

            dfs(nr, nc, ne, nmask, moves + 1);
        }
    }

    int minMoves(vector<string>& classroom, int energy) {
        g = classroom;
        m = g.size(); n = g[0].size();
        energyCap = energy;
        bit.assign(m, vector<int>(n, -1));

        int sr = 0, sc = 0, cnt = 0;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                if (g[i][j] == 'S') { sr = i; sc = j; }
                else if (g[i][j] == 'L') bit[i][j] = cnt++;
            }
    }
};
```



```

        if (cnt == 0) return 0;
        full = (1 << cnt) - 1;
        best = INT_MAX;

        dfs(sr, sc, energy, 0, 0);
        return best == INT_MAX ? -1 : best;
    }
};

int main() {
    vector<string> classroom = {"LS", "RL"};
    int energy = 4;
    Solution sol;
    cout << sol.minMoves(classroom, energy) << endl;    // Expected output: 3
    return 0;
}

```

■ Python

```

import sys

class Solution:
    def minMoves(self, classroom, energy):
        self.g = classroom
        self.m, self.n = len(classroom), len(classroom[0])
        self.energyCap = energy
        self.bit = [[-1] * self.n for _ in range(self.m)]

        sr = sc = cnt = 0
        for i in range(self.m):
            for j in range(self.n):
                if self.g[i][j] == 'S':
                    sr, sc = i, j
                elif self.g[i][j] == 'L':
                    self.bit[i][j] = cnt
                    cnt += 1

        if cnt == 0:
            return 0
        self.full = (1 << cnt) - 1
        self.best = sys.maxsize
        self.dfs(sr, sc, energy, 0, 0)
        return -1 if self.best == sys.maxsize else self.best

    def dfs(self, r, c, e, mask, moves):
        if moves >= self.best:
            return
        if mask == self.full:
            self.best = min(self.best, moves)
            return
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if not (0 <= nr < self.m and 0 <= nc < self.n):
                continue
            if self.g[nr][nc] == 'X':
                continue
            if e == 0:
                continue
            ne = e - 1
            nmask = mask
            if self.bit[nr][nc] != -1:
                nmask |= (1 << self.bit[nr][nc])
            if self.g[nr][nc] == 'R':
                ne = self.energyCap
            self.dfs(nr, nc, ne, nmask, moves + 1)

if __name__ == "__main__":
    classroom = ["LS", "RL"]
    energy = 4
    print(Solution().minMoves(classroom, energy))    # Expected output: 3

```

```
class Solution {
  minMoves(classroom, energy) {
    const m = classroom.length, n = classroom[0].length;
    const bit = Array.from({ length: m }, () => Array(n).fill(-1));
    let sr = 0, sc = 0, cnt = 0;
    for (let i = 0; i < m; i++)
      for (let j = 0; j < n; j++) {
        if (classroom[i][j] === 'S') { sr = i; sc = j; }
        else if (classroom[i][j] === 'L') bit[i][j] = cnt++;
      }
    if (cnt === 0) return 0;

    const full = (1 << cnt) - 1;
    let best = Infinity;
    const dr = [-1, 1, 0, 0], dc = [0, 0, -1, 1];

    const dfs = (r, c, e, mask, moves) => {
      if (moves >= best) return;
      if (mask === full) { best = Math.min(best, moves); return; }
      for (let d = 0; d < 4; d++) {
        const nr = r + dr[d], nc = c + dc[d];
        if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
        if (classroom[nr][nc] === 'X') continue;
        if (e === 0) continue;
        let ne = e - 1;
        let nmask = mask;
        if (bit[nr][nc] !== -1) nmask |= (1 << bit[nr][nc]);
        if (classroom[nr][nc] === 'R') ne = energy;
        dfs(nr, nc, ne, nmask, moves + 1);
      }
    };

    dfs(sr, sc, energy, 0, 0);
    return best === Infinity ? -1 : best;
  }
}

// Example usage
const classroom = ["LS", "RL"];
const energy = 4;
console.log(new Solution().minMoves(classroom, energy)); // Expected output: 3
```

Complexity: Time is roughly $O(4^{\text{moves}})$ in the worst case — exponential, because the same (cell, energy, mask) situation can be re-explored endlessly along different paths. Space is $O(\text{moves})$ for the recursion stack. This works fine on tiny grids like the examples but will time out on the full constraint range (20×20 grid, energy up to 50).

Approach 2 — Better (BFS Over the Full State Space)

Idea: Fix the brute force's core flaw — re-exploring the same situation repeatedly — by remembering every (row, col, energy, mask) combination we've already queued, and switch from DFS to **BFS** so the first time we reach a full mask is guaranteed to be the minimum number of moves.

Steps

- Parse the grid the same way: find 'S', assign bit indices to each 'L'.
- Create a 4-D visited array **vis[row][col][energy][mask]** so no situation is expanded twice.
- Push the start state (sr, sc, energy, mask=0) into a queue and mark it visited.
- Run standard level-order BFS: for every state popped, try all 4 directions, apply the same energy/reset/collect rules as before, and push any new, not-yet-visited state.
- As soon as a popped state has mask == full, the current BFS level number is the answer.
- If the queue empties out first, return -1.

Approach 2 — Code (all languages run Example 2)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minMoves(vector<string>& classroom, int energy) {
        int m = classroom.size(), n = classroom[0].size();
        vector<vector<int>> bit(m, vector<int>(n, -1));

        int sr = 0, sc = 0, cnt = 0;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                if (classroom[i][j] == 'S') { sr = i; sc = j; }
                else if (classroom[i][j] == 'L') bit[i][j] = cnt++;
            }
        if (cnt == 0) return 0;

        int full = (1 << cnt) - 1;
        int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};

        // vis[row][col][energy][mask]
        vector<vector<vector<vector<bool>>>> vis(
            m, vector<vector<vector<bool>>>(
                n, vector<vector<bool>>(
                    energy + 1, vector<bool>(1 << cnt, false))));

        queue<array<int, 4>> q; // {row, col, energy, mask}
        q.push({sr, sc, energy, 0});
        vis[sr][sc][energy][0] = true;
        int moves = 0;

        while (!q.empty()) {
            int sz = q.size();
            while (sz--) {
                auto [r, c, e, mask] = q.front(); q.pop();
                if (mask == full) return moves;
                if (e == 0) continue; // no energy, not moving further

                for (int d = 0; d < 4; d++) {
                    int nr = r + dr[d], nc = c + dc[d];
                    if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
                    if (classroom[nr][nc] == 'X') continue;

                    int ne = e - 1;
```



```

        int nmask = mask;
        if (bit[nr][nc] != -1) nmask |= (1 << bit[nr][nc]);
        if (classroom[nr][nc] == 'R') ne = energy;

        if (!vis[nr][nc][ne][nmask]) {
            vis[nr][nc][ne][nmask] = true;
            q.push({nr, nc, ne, nmask});
        }
    }
    moves++;
}
return -1;
}
};

int main() {
    vector<string> classroom = {"LS", "RL"};
    int energy = 4;
    Solution sol;
    cout << sol.minMoves(classroom, energy) << endl;    // Expected output: 3
    return 0;
}

```

■ Python

```

from collections import deque

class Solution:
    def minMoves(self, classroom, energy):
        m, n = len(classroom), len(classroom[0])
        bit = [[-1] * n for _ in range(m)]

        sr = sc = cnt = 0
        for i in range(m):
            for j in range(n):
                if classroom[i][j] == 'S':
                    sr, sc = i, j
                elif classroom[i][j] == 'L':
                    bit[i][j] = cnt
                    cnt += 1

        if cnt == 0:
            return 0

        full = (1 << cnt) - 1
        vis = [[[False] * (1 << cnt) for _ in range(energy + 1)]
                for _ in range(n)] for _ in range(m)]

        q = deque([(sr, sc, energy, 0)])
        vis[sr][sc][energy][0] = True
        moves = 0

        while q:
            for _ in range(len(q)):
                r, c, e, mask = q.popleft()
                if mask == full:
                    return moves
                if e == 0:
                    continue
                for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                    nr, nc = r + dr, c + dc
                    if not (0 <= nr < m and 0 <= nc < n):
                        continue
                    if classroom[nr][nc] == 'X':
                        continue
                    ne = e - 1
                    nmask = mask
                    if bit[nr][nc] != -1:
                        nmask |= (1 << bit[nr][nc])
                    if classroom[nr][nc] == 'R':
                        ne = energy

```

```

        if not vis[nr][nc][ne][nmask]:
            vis[nr][nc][ne][nmask] = True
            q.append((nr, nc, ne, nmask))

        moves += 1
    return -1

if __name__ == "__main__":
    classroom = ["LS", "RL"]
    energy = 4
    print(Solution().minMoves(classroom, energy))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    minMoves(classroom, energy) {
        const m = classroom.length, n = classroom[0].length;
        const bit = Array.from({ length: m }, () => Array(n).fill(-1));
        let sr = 0, sc = 0, cnt = 0;
        for (let i = 0; i < m; i++)
            for (let j = 0; j < n; j++) {
                if (classroom[i][j] === 'S') { sr = i; sc = j; }
                else if (classroom[i][j] === 'L') bit[i][j] = cnt++;
            }
        if (cnt === 0) return 0;

        const full = (1 << cnt) - 1;
        const dr = [-1, 1, 0, 0], dc = [0, 0, -1, 1];
        const MASK = 1 << cnt;
        const vis = new Uint8Array(m * n * (energy + 1) * MASK);
        const idx = (r, c, e, mask) => ((r * n + c) * (energy + 1) + e) * MASK + mask;

        let queue = [[sr, sc, energy, 0]];
        vis[idx(sr, sc, energy, 0)] = 1;
        let moves = 0;

        while (queue.length) {
            const next = [];
            for (const [r, c, e, mask] of queue) {
                if (mask === full) return moves;
                if (e === 0) continue;
                for (let d = 0; d < 4; d++) {
                    const nr = r + dr[d], nc = c + dc[d];
                    if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
                    if (classroom[nr][nc] === 'X') continue;
                    let ne = e - 1;
                    let nmask = mask;
                    if (bit[nr][nc] !== -1) nmask |= (1 << bit[nr][nc]);
                    if (classroom[nr][nc] === 'R') ne = energy;
                    const id = idx(nr, nc, ne, nmask);
                    if (!vis[id]) { vis[id] = 1; next.push([nr, nc, ne, nmask]); }
                }
            }
            queue = next;
            moves++;
        }
        return -1;
    }
}

const classroom = ["LS", "RL"];
const energy = 4;
console.log(new Solution().minMoves(classroom, energy)); // Expected output: 3

```

Complexity: Each state (row, col, energy, mask) is visited at most once, so time and space are both $O(m \times n \times \text{energy} \times 2^L)$, where $L \leq 10$ is the number of litter cells. For the largest inputs this is about $20 \times 20 \times 51 \times 1024 \approx 20.9$ million states — a huge improvement over exponential brute force, and this is already the intended solution for this problem.

Approach 3 — Optimal (Flattened State BFS + Early Exit)

Idea: Keep the exact same correct algorithm as Approach 2 (there is no lower complexity class for this problem — it's inherently a search over that 4-part state), but tighten the implementation so it runs with much better constants and less memory: encode the whole state as a single integer instead of a nested 4-D structure, use a flat byte array instead of nested boolean containers (fewer allocations, better cache locality), and check for a completed mask the moment it's produced instead of waiting for it to be dequeued a level later.

Steps

- Parse the grid into a flat bit-index array (`bitOf[row*n+col]`) instead of a 2-D structure.
- Pick $E = \text{energy} + 1$ and $\text{MASK} = 1 \ll \text{cnt}$, and pack a state into one integer: $\text{code} = ((\text{row} * n + \text{col}) * E + \text{energy}) * \text{MASK} + \text{mask}$.
- Use a single flat byte array of size $m * n * E * \text{MASK}$ as the visited table — one contiguous block instead of 4 nested containers.
- Run BFS level by level using two plain arrays (current / next) instead of a queue object — avoids per-element queue overhead.
- The moment a neighbour transition produces $\text{nmask} == \text{full}$, return $\text{moves} + 1$ immediately — no need to wait to dequeue that state on the next level.
- If both frontiers become empty without ever completing the mask, return -1.

Approach 3 — Code (all languages run Example 2)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minMoves(vector<string>& classroom, int energy) {
        int m = classroom.size(), n = classroom[0].size();
        vector<int> bitOf(m * n, -1);

        int sr = 0, sc = 0, cnt = 0;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                if (classroom[i][j] == 'S') { sr = i; sc = j; }
                else if (classroom[i][j] == 'L') bitOf[i * n + j] = cnt++;
            }
        if (cnt == 0) return 0;

        int full = (1 << cnt) - 1;
        int E = energy + 1, MASK = 1 << cnt;
        long long total = (long long)m * n * E * MASK;
        vector<char> vis(total, 0); // flat visited table

        auto encode = [&](int r, int c, int e, int mask) {
            return ((r * n + c) * E + e) * MASK + mask;
        };

        vector<int> cur = { encode(sr, sc, energy, 0) };
        vis[cur[0]] = 1;
        int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
        int moves = 0;

        while (!cur.empty()) {
            vector<int> nxt;
            nxt.reserve(cur.size() * 3);

            for (int code : cur) {
                int mask = code % MASK;
```

```

        int t = code / MASK;
        int e = t % E;
        int pos = t / E;
        int r = pos / n, c = pos % n;
        if (e == 0) continue;           // stuck: out of energy

        for (int d = 0; d < 4; d++) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
            if (classroom[nr][nc] == 'X') continue;

            int ne = e - 1;
            int nmask = mask;
            int b = bitOf[nr * n + nc];
            if (b != -1) nmask |= (1 << b);
            if (classroom[nr][nc] == 'R') ne = energy;

            if (nmask == full) return moves + 1;    // early exit

            int code2 = encode(nr, nc, ne, nmask);
            if (!vis[code2]) { vis[code2] = 1; nxt.push_back(code2); }
        }
        cur.swap(nxt);
        moves++;
    }
    return -1;
}

};

int main() {
    vector<string> classroom = {"LS", "RL"};
    int energy = 4;
    Solution sol;
    cout << sol.minMoves(classroom, energy) << endl;    // Expected output: 3
    return 0;
}

```

■ Python

```

class Solution:
    def minMoves(self, classroom, energy):
        m, n = len(classroom), len(classroom[0])
        bitOf = [-1] * (m * n)

        sr = sc = cnt = 0
        for i in range(m):
            for j in range(n):
                if classroom[i][j] == 'S':
                    sr, sc = i, j
                elif classroom[i][j] == 'L':
                    bitOf[i * n + j] = cnt
                    cnt += 1

        if cnt == 0:
            return 0

        full = (1 << cnt) - 1
        E, MASK = energy + 1, 1 << cnt

        def encode(r, c, e, mask):
            return ((r * n + c) * E + e) * MASK + mask

        vis = bytearray(m * n * E * MASK)
        cur = [encode(sr, sc, energy, 0)]
        vis[cur[0]] = 1
        dr, dc = (-1, 1, 0, 0), (0, 0, -1, 1)
        moves = 0

        while cur:
            nxt = []
            for code in cur:
                mask = code % MASK

```

```

        t = code // MASK
        e = t % E
        pos = t // E
        r, c = divmod(pos, n)
        if e == 0:
            continue
        for k in range(4):
            nr, nc = r + dr[k], c + dc[k]
            if not (0 <= nr < m and 0 <= nc < n):
                continue
            if classroom[nr][nc] == 'X':
                continue
            ne = e - 1
            nmask = mask
            b = bitOf[nr * n + nc]
            if b != -1:
                nmask |= (1 << b)
            if classroom[nr][nc] == 'R':
                ne = energy
            if nmask == full:
                return moves + 1
            code2 = encode(nr, nc, ne, nmask)
            if not vis[code2]:
                vis[code2] = 1
                nxt.append(code2)

        cur = nxt
        moves += 1
    return -1

if __name__ == "__main__":
    classroom = ["LS", "RL"]
    energy = 4
    print(Solution().minMoves(classroom, energy))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    minMoves(classroom, energy) {
        const m = classroom.length, n = classroom[0].length;
        const bitOf = new Int16Array(m * n).fill(-1);
        let sr = 0, sc = 0, cnt = 0;
        for (let i = 0; i < m; i++)
            for (let j = 0; j < n; j++) {
                if (classroom[i][j] === 'S') { sr = i; sc = j; }
                else if (classroom[i][j] === 'L') bitOf[i * n + j] = cnt++;
            }
        if (cnt === 0) return 0;

        const full = (1 << cnt) - 1;
        const E = energy + 1, MASK = 1 << cnt;
        const encode = (r, c, e, mask) => ((r * n + c) * E + e) * MASK + mask;
        const vis = new Uint8Array(m * n * E * MASK);

        let cur = [encode(sr, sc, energy, 0)];
        vis[cur[0]] = 1;
        const dr = [-1, 1, 0, 0], dc = [0, 0, -1, 1];
        let moves = 0;

        while (cur.length) {
            const nxt = [];
            for (const code of cur) {
                const mask = code % MASK;
                const t = Math.floor(code / MASK);
                const e = t % E;
                const pos = Math.floor(t / E);
                const r = Math.floor(pos / n), c = pos % n;
                if (e === 0) continue;
                for (let d = 0; d < 4; d++) {
                    const nr = r + dr[d], nc = c + dc[d];
                    if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
                    if (classroom[nr][nc] === 'X') continue;

```

```

        let ne = e - 1;
        let nmask = mask;
        const b = bitOf[nr * n + nc];
        if (b !== -1) nmask |= (1 << b);
        if (classroom[nr][nc] === 'R') ne = energy;
        if (nmask === full) return moves + 1;
        const code2 = encode(nr, nc, ne, nmask);
        if (!vis[code2]) { vis[code2] = 1; nxt.push(code2); }
    }
}
cur = nxt;
moves++;
}
return -1;
}
}

const classroom = ["LS", "RL"];
const energy = 4;
console.log(new Solution().minMoves(classroom, energy)); // Expected output: 3

```

Complexity: Same asymptotic bound as Approach 2, $O(m \times n \times \text{energy} \times 2^L)$ time and space — but in practice noticeably faster and lighter, because it avoids the allocation and pointer-chasing overhead of deeply nested boolean-vector visited structures, and it can return the moment a full mask is first produced rather than waiting for the next BFS level to dequeue it.

Side-by-Side Comparison

1. Brute Force	DFS over all paths, no visited tracking	$O(4^{\text{moves}})$ — exponential	Optimal
2. Better	BFS over (row, col, energy, mask) with 4-D visited array	$O(4 \cdot n \cdot \text{energy} \cdot 2^m)$	Optimal
3. Optimal	Same BFS, flattened integer state + early exit on full mask	$O(n \cdot \text{energy} \cdot 2^m)$	Optimal

Key Takeaways to Remember

- **"At most 10" of something** in constraints is a strong hint for a **bitmask** in the state.
- When a grid problem has a resource that can run out and be refilled at special cells, that resource almost always needs to become **part of the BFS state**, not a separate check.
- Because every move costs the same (1 unit), plain level-order **BFS** already finds the minimum — there's no need for Dijkstra/priority-queue machinery here.
- Going from "better" to "optimal" here is about **implementation efficiency** (flat arrays, integer encoding, early exit), not a different algorithmic idea — that's a common and realistic pattern in state-space search problems.
- All three tiers were validated against **Example 2** (classroom = ["LS","RL"], energy = 4 → 3) across C++, Python, and JavaScript.